

Informe Tarea 1

Matías Acevedo, macevedoa@usm.cl

Vicente Currumilla, vcurremilla@usm.cl

Vicente Flores A, vfloresa@usm.cl

Roberto Salazar, rsalazarf@usm.cl

Vicente Sánchez, vsanchezg@usm.cl

Vicente Zárate, vzarate@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

La función lecturaSensor() tiene como objetivo simular la obtención de una medida de voltaje de un sensor. Para esto, crea una variable llamada lecturaAleatoria tipo double, para luego asignarle un valor aleatorio a través de la función rand() % 256, lo que significa que elegirá un valor entre 0 y 255, terminando con el retorno de la variable lecturaAleatoria y el valor ya asignado.

Por otra parte, se implementa el enum Accion, para definir los tres posibles procesos de uso de los sensores, los cuales son: INIT, READ y SHOW. Así, al aplicar un switch(), el código es más fácil de entender y permite que la función usarSensores() sepa exactamente qué hacer en cada caso, es decir, si debe iniciar las mediciones asignando el valor cero (caso INIT), realizar las lecturas aleatorias con la función lecturaSensor() para asignar un valor entre 0 y 255 (caso READ), o mostrar las mediciones en pantalla de la etapa anterior (caso SHOW).

En la Figura 1 se muestra el diagrama de flujo de las funciones lecturaSensor() y usarSensores() en conjunto. El algoritmo consiste en gestionar un grupo de sensores mediante tres acciones ya definidas. Para ello se recibe una lista de sensores y una acción a realizar, la cual puede ser; iniciar los sensores desde cero, leer y guardar el valor de voltaje del sensor, o mostrar los valores de los sensores.

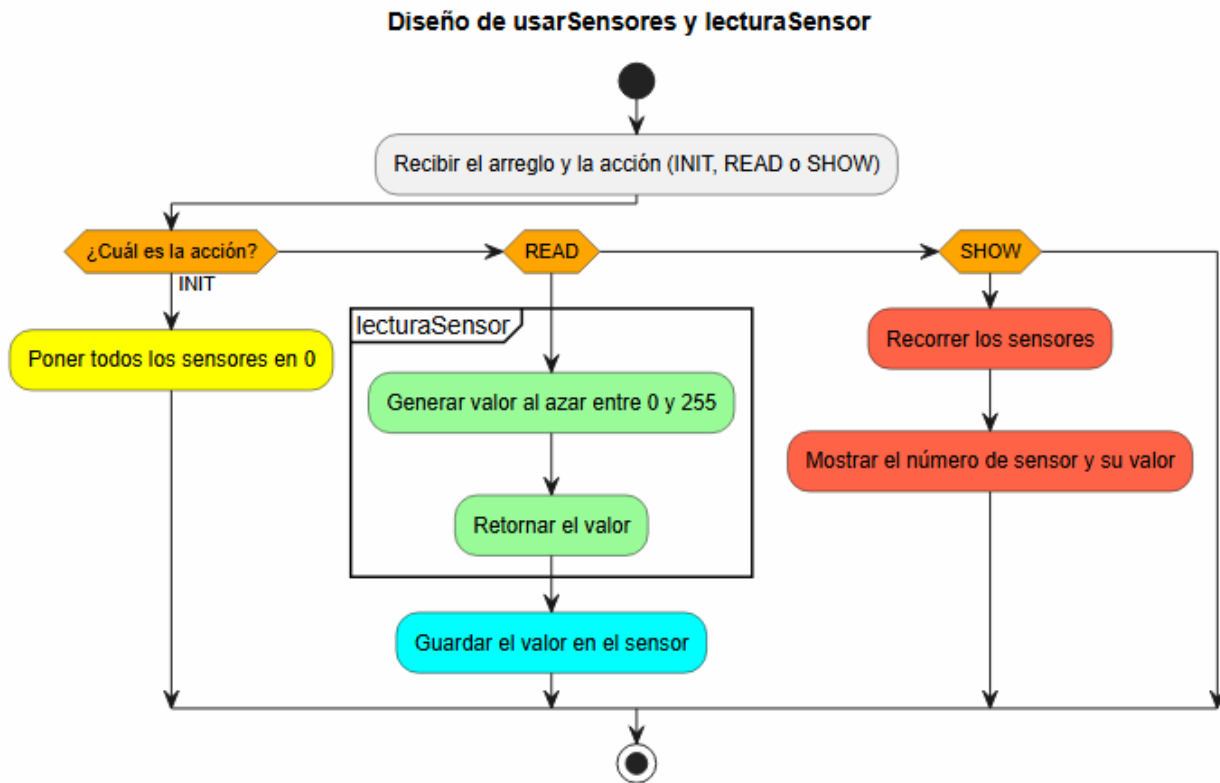


Figura 1

1.2. Diseño completo de la nueva función main()

A diferencia de la versión anterior donde todas las funciones estaban apiladas, ahora el main() es mucho más simple. Primero crea el arreglo para los sensores y luego llama a la nueva función usarSensores() pasándole el nombre de la acción que debe realizar para cada caso. De esta forma, el código queda mucho más ordenado y cada tarea (inicializar, leer y mostrar) se ejecuta de manera independiente. Esto se muestra en el diagrama de flujo de la Figura 2:

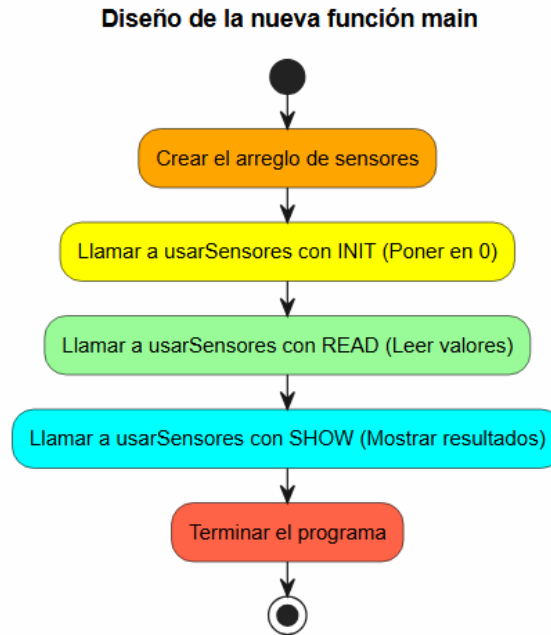


Figura 2

2. Parte 2

2.1. Hito 1

1. Qué es lo que retorna la función `clock()`.

La función retorna la cantidad transcurrida de una unidad de tiempo, en este caso, al investigar nos dimos cuenta que esta unidad son ticks o ciclos de reloj de la CPU. Esto lo concluimos porque para calcular el tiempo total del for, se determina la cantidad de ticks que demoró el ciclo con la diferencia (fin - inicio). Este resultado se divide por la constante `CLOCKS_PER_SEC` para transformar los ticks en segundos y finalmente se multiplica por $1e9$ para convertir esos segundos en nanosegundos.

2. Qué unidades tienen las variables inicio y fin.

Las variables inicio y fin están medidas en ticks, ya que guardan lo que retorna la función `clock()` en esos instantes. En este caso, descubrimos que para guardar los ticks, se usa el tipo de dato `clock_t` de la librería `<time.h>`.

3. Por qué para obtener el tiempo total de duración del ciclo for se debe dividir la diferencia entre fin e inicio con el valor constante `CLOCKS_PER_SEC`.

Como analizamos anteriormente, la diferencia entre fin e inicio determina la cantidad de ticks ocurridos durante el ciclo for, por lo tanto, esto hay que dividirlo por la constante `CLOCKS_PER_SEC` para obtener el tiempo en segundos del ciclo.

4. Explicar que hace (double) como prefijo de (fin - inicio))

Lo que hace el prefijo double es convertir explícitamente la cantidad de ticks del ciclo; de un entero de 64 bits a un dato tipo double.

5. Explicar para que sirve utilizar (double) como prefijo de (fin - inicio))

Esto sirve para que al dividir la cantidad de ticks por la constante, C detecte que el resultado de la división debe ser un punto flotante y no se pierda precisión en el cálculo del tiempo total en nanosegundos.

2.2. Hito 2

En la Figura 3 se muestra el diagrama de flujo de la función cicloPrueba(). El algoritmo consiste en medir el tiempo promedio que tarda el procesador en realizar ciclos vacíos, mostrar espacios, asteriscos o saltos de líneas una cierta cantidad de veces, según el caso que se le indique. Para esto, se guarda el tiempo inicial, ejecuta el ciclo correspondiente y, tras obtener el tiempo final, calcula el promedio en nanosegundos utilizando una conversión para asegurar la precisión de los resultados.

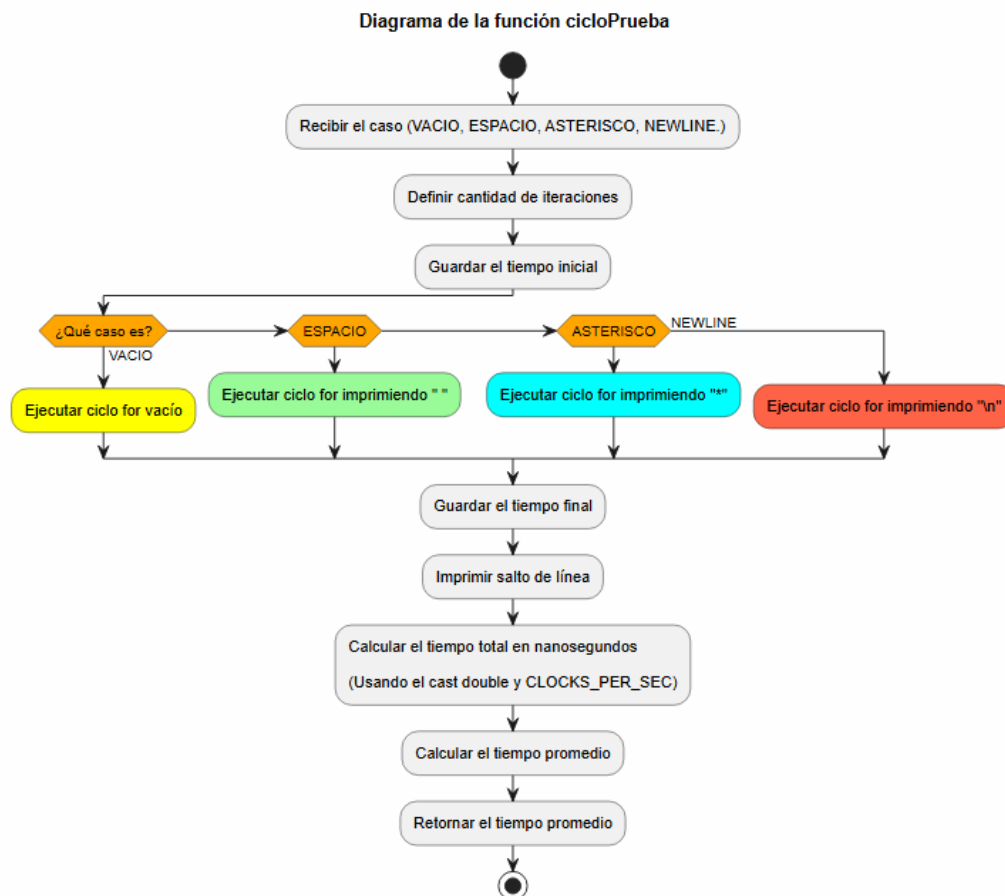


Figura 3

2.3. Hito 3

Para medir los tiempos de ejecución se usó el archivo mediciones.cc en VSCode. Se compiló y ejecutó 10 veces para obtener una muestra y poder analizar los resultados.

Para esto se utilizó un computador con un procesador Intel(R) Core(TM) i5-10300H (2.50 GHz), 8 GB de memoria RAM DDR4 (2933 MT/s) y sistema operativo Windows 11 Home de 64 bits. Además, se usó el compilador gcc.exe (Rev13, Built by MSYS2 project) versión 15.2.0 y al igual que en la parte 1, se modificó el archivo makefile para poder ejecutar el código en el equipo ya que no nos dejaba: se cambió la línea del makefile aportado desde las indicaciones “@mkdir -p \$(DIR_OBJ)”, por “@if not exist \$(DIR_OBJ) mkdir \$(DIR_OBJ)”. Y en vez de colocar “make” en la terminal, en este caso se le debe colocar “mingw32-make”

Los resultados obtenidos en nanosegundos se muestran en la Tabla 1:

Caso	tpro m 1	tpro m 2	tpro m 3	tpro m 4	tpro m 5	tpro m 6	tpro m 7	tpro m 8	tpro m 9	tpro m 10
VACIO	2	3	5	2	2	2	2	2	4	2
ESPACIO	31087	30881	31185	31458	31918	31208	31554	31656	31729	30804
ASTERISCO	31487	31428	32651	31345	32296	31346	31553	31904	31396	31282
NEWLINE	28639	30685	28947	28625	28563	29108	29830	28442	28357	29070

Tabla 1

De las mediciones anteriores se realizó el cálculo del promedio y desviación estándar de cada caso, lo cual se muestra en la Tabla 2:

Caso	Promedio	Desv. Estándar
VACIO	2.6	0.97
ESPACIO	31.348	373.18
ASTERISCO	31,668.8	444.60
NEWLINE	29,026.6	694.04

Tabla 2

Al analizar los datos, nos damos cuenta que realizar ciclos vacíos es mucho más rápido (alrededor de 11.000 veces más rápido) que los demás casos donde los ciclos de Entrada/Salida deben imprimir caracteres en la terminal. Además, las pequeñas variaciones en las mediciones nos indican que puede haber ciertos factores que causan estas diferencias, como tareas secundarias que realicen el procesador y el sistema operativo mientras se ejecuta el código. Esto también explica por qué los resultados del ciclo vacío casi no tienen variación, mientras que al imprimir caracteres hay más diferencia porque el computador realiza procesos más complejos y lentos.